Desarrollo de Interfaces

Confección de interfaces

Tema 1

---

1. Introducción

Interfaz de usuario Elemento de una aplicación que permite al usuario comunicarse con ella. Las hay **textuales** (órdenes escritas en un intérprete), **gráficas** (iconos, menús y ventanas) y **táctiles** (la pantalla reacciona al toque).

Componente / control Elemento gráfico atómico con el que el usuario da órdenes o recibe información. Separados entre **pasivos** (muestran información) y **activos** (reciben datos del usuario).

2. Herramientas y librerías

Librería Conjunto de **clases** que se incluyen el proyecto para **construir interfaces.**

Algunas de las disponibles en Java son:

- **AWT** Usa componentes nativos. **En desuso.**
- **Swing** Componentes ligeros escritos en Java. **En mantenimiento.**
- **JavaFX** Sucesora de Swing. Activa en el proyecto **OpenJFX.**

3. Arquitectura de JavaFX

- **Stage** Ventana del sistema operativo.
- **Scene** Envuelve el nodo raíz: contiene **tamaño, hojas de estilo** y **aceleradores.**
- **Node** Unidad básica del árbol visual.
- **Parent** Nodo que puede tener hijos.
- **Region** Subclase de Parent con tamaño y soporte para CSS.
- **Control** Componente con comportamiento y skin.

3.1. Ciclo de vida

- **main()** llama a **Application.launch** para iniciar JavaFX.
- **init()** Paso opcional que se ejecuta antes del hilo de JavaFX. **No puede tocar la interfaz.**
- **start()** Se ejecuta en el **hilo de aplicación** para crear la interfaz.
- **stop()** Llamado al cerrar para liberar recursos.

4. Contenedores y ubicación de componentes

Contenedor Parent que coloca y dimensiona automáticamente a sus hijos. Permite interfaces adaptables al redimensionar la ventana.

Contenedor	Descripción
Pane	Distribución absoluta, sin reglas
HBox	Fila horizontal
VBox	Columna vertical
BorderPane	Compuesto por 5 zonas
GridPane	Filas y columnas, con celdas combinables.
StackPane	Nodos superpuestos y centrados.
FlowPane	Fluye y salta de línea
TilePane	Rejilla de celdas iguales
AnchorPane	Ancla componentes a los bordes
Group	Coordenadas reales
TabPane	Pestañas
ScrollPane	Incluye barras de desplazamiento automáticas
SplitPane	Dos o más zonas separadas por divisores móviles
Accordion / TitledPane	Paneles plegables
ToolBar	Barra de herramientas
MenuBar	Barra de menús

4.1. Propiedades

Cada propiedad de un componente es un **objeto observable** que permite escuchar cambios y enlazarlos a listeners.

```
1 campoTexto.textProperty().addListener(observable,valorAnterior,valorNuevo)
```

4.2. Estilos

Los componentes pueden tener un estilo mediante CSS, con propiedades que empiezan por `-fx-`. Pueden hacerse **en línea** (propiedad `style` del nodo), por **clase** (`nodo.getStyleClass().add()`) o en una **hoja de estilos.**

5. Ejemplos de código

5.1. Cambiar la Scene de una Stage

```
1  // Dentro de start(Stage stage)
2  Button ir = new Button("Ir a escena 2");
3  StackPane pane1 = new StackPane(new Label("Escena 1"), ir);
4  Scene scene1 = new Scene(pane1, 600, 400);
5  Button volver = new Button("Volver a escena 1");
6  StackPane pane2 = new StackPane(new Label("Escena 2"),
7  volver);
8  Scene scene2 = new Scene(pane2, 600, 400);
9  ir.setOnAction(e -> stage.setScene(scene2));
10 volver.setOnAction(e -> stage.setScene(scene1));
11 stage.setScene(scene1);
12 stage.show();
```

5.2. Sustituir la raíz de una Scene

```
1 StackPane root1 = new StackPane(new Label("Root 1"));
2 StackPane root2 = new StackPane(new Label("Root 2"));
3 Scene scene = new Scene(root1, 800, 600);
4 stage.setScene(scene);
5 // cambiar root sin crear nueva Scene
6 root1.getChildren().add(new Button("Ir") {{
7 setOnAction(ev -> scene.setRoot(root2));
8 }});
```

5.3. Múltiples hijos en la misma ventana

```
1 StackPane stack = new StackPane();
2 BorderPane screenHome = new BorderPane(new Label("Home"));
3 BorderPane screenSettings = new BorderPane(new
4 Label("Settings"));
5 stack.getChildren().addAll(screenHome, screenSettings);
6 screenSettings.setVisible(false);
7 Button goSettings = new Button("Settings");
```

```
8  goSettings.setOnAction(e -> {
9  screenSettings.setVisible(true);
10 screenSettings.toFront();
```

5.4. Ventana modal

```
1  Stage dialog = new Stage();
2  dialog.initOwner(primaryStage);
3  dialog.initModality(Modality.APPLICATION_MODAL); // bloquea
4  owner esta sera modal y no permitira que se acceda a la
5  ventana principal hasta que no se finalice.
6  dialog.setScene(new Scene(new VBox(new Label("Confirmar...")),
7  250, 150));
8  dialog.showAndWait(); // bloquea el hilo que llamo hasta
9  cerrar el dialog
```

5.5. Obtener el controlador de una escena

```
1  FXMLLoader loader = new FXMLLoader(App.class.getResource("detalle-view.fxml"));
2  Parent root = loader.load();
3  DetalleController ctrl = loader.getController();
4  ctrl.setPersona(personaSeleccionada);
```

© 2026 - Esteban N. - estnicolas@proton.me

Este texto se encuentra licenciado bajo Creative Commons Atribución-Reconocimiento/NoComercial-CompartirIgual 4.0 Internacional.
Para ver una copia de esta licencia visite:
https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.es.



Ninguna parte de este documento, código o texto, ha sido realizada, apoyada, consultada, generada, corregida o involucrada en ninguna forma o contexto con inteligencia artificial, ni lo será en cualquier futura posible versión o corrección.
https://notbyai.fyi